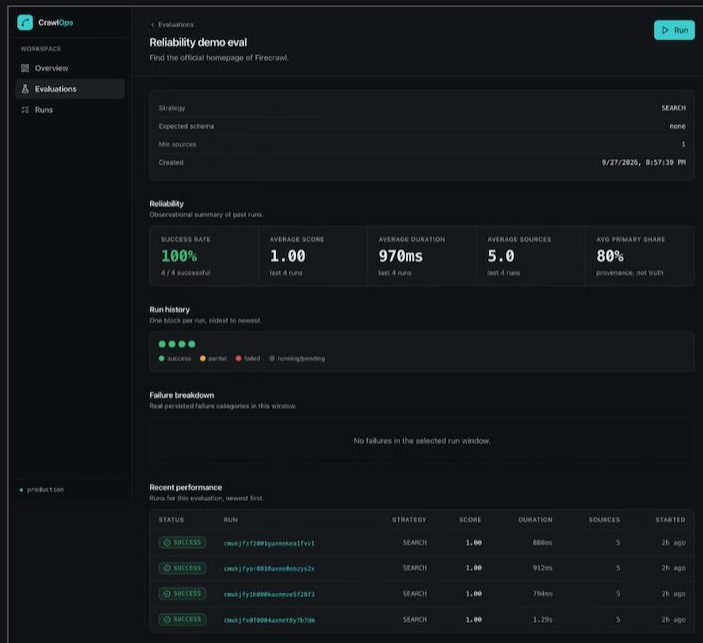


```
C>_ /crawlops/init
```

CRAWLOPS

RELIABILITY INFRASTRUCTURE FOR WEB RESEARCH WORKFLOWS

Built on Firecrawl to validate output contracts,
preserve provenance, diagnose failures, and
measure repeated-run reliability.



```
engine: Firecrawl · layer: evaluator · target: contract-  
satisfaction
```

API SUCCESS $\neq$ WORKFLOW SUCCESS

PROVIDER LAYER

**Provider: Firecrawl
request completed ✓**

HTTP 200 OK · Network transport intact ·
Upstream crawl job executed

APPLICATION LAYER

**Application workflow: Empty output ·
Schema mismatch · Insufficient
sources · Weak provenance · Rate
limit · Unexpected result**

Contract satisfaction · Source threshold audit · Structured field
integrity

Crawl0ps evaluates whether the research actually satisfied the application's contract.

RESEARCH → EVALUATE → PERSIST → MEASURE

01 Task + JSON Schema Contract spec	02 SEARCH / AGENT strategy Execution mode	03 Firecrawl Web research	04 Output + sources Raw artifact
05 Deterministic evaluator Rule validation	06 SUCCESS / PARTIAL / FAILED Status verdict	07 PostgreSQL history State storage	08 Reliability analytics Repeated metrics
PRODUCTION Containerized Azure deployment	Azure Container Apps • React/nginx frontend container • Fastify API container Azure PostgreSQL Flexible Server · Private Azure Container Registry · Scale-to-zero · health/readiness probes · Dockerized deployment		

PROVENANCE TRACE: For AGENT runs, CrawlOps captures fetched-page provenance from Firecrawl's execution trace.

EVERY RUN IS EXPLAINABLE

- Structured output validation against JSON Schema contracts
- Deterministic checks across output and persisted sources
- Attempts and retries lifecycle tracking
- Persisted source URLs and fetched-page provenance
- Explicit failure categories

The screenshot displays the CrawlOps interface for an audit run. On the left, a sidebar shows the 'Workspace' with 'Overview', 'Evaluations', and 'Runs' (selected). The main panel shows the audit results for a 'production' run.

Workspace

- Overview
- Evaluations
- Runs

production

01 Produced a non-empty result PASS critical
Output is present.

02 All source URLs are valid PASS
All source URLs are well-formed http(s) URLs.

03 No duplicate sources PASS
No duplicate source URLs.

04 Attempts 1 total

Attempt #1 • SEARCH 902ms SUCCESS
1 Firecrawl call(s)

05 Sources 5 retrieved

SOURCE QUALITY				provenance estimate, not truth
Primary	Secondary	Community	Unknown	Primary share
4	0	1	0	80%

01 www.firecrawl.dev PRIMARY Open
Firecrawl - The web data API to search, scrape, and interact with the web ...
<https://www.firecrawl.dev/>
Power AI agents with clean web data. The web data API to search, scrape, and interact at scale. It's also open source. Start for free. Setup for agents.Pricing · Docs · Blog · Playground

02 [github.com](https://github.com/firecrawl/firecrawl) PRIMARY Open
GitHub - firecrawl/firecrawl: The web data API to search, scrape, and ...
<https://github.com/firecrawl/firecrawl>
** 🚀 Firecrawl** [Permalink: 🌟 Firecrawl](https://github.com/firecrawl/firecrawl/#-firecrawl) ## Quick Start ### Search ``` firecrawl search "firecrawl" --limit 5 ``` ### Scrape ``` from firecrawl import Firecrawl app = Firecrawl(api_key="fc-YOUR_API_KEY") result = app.scrape("firecrawl.dev") ```
[Show more](#)

03 www.firecrawl.dev PRIMARY Open
The web data API to search, scrape, and interact with the web ... - Firecrawl

NOTE: Source authority estimates provenance – not factual truth.

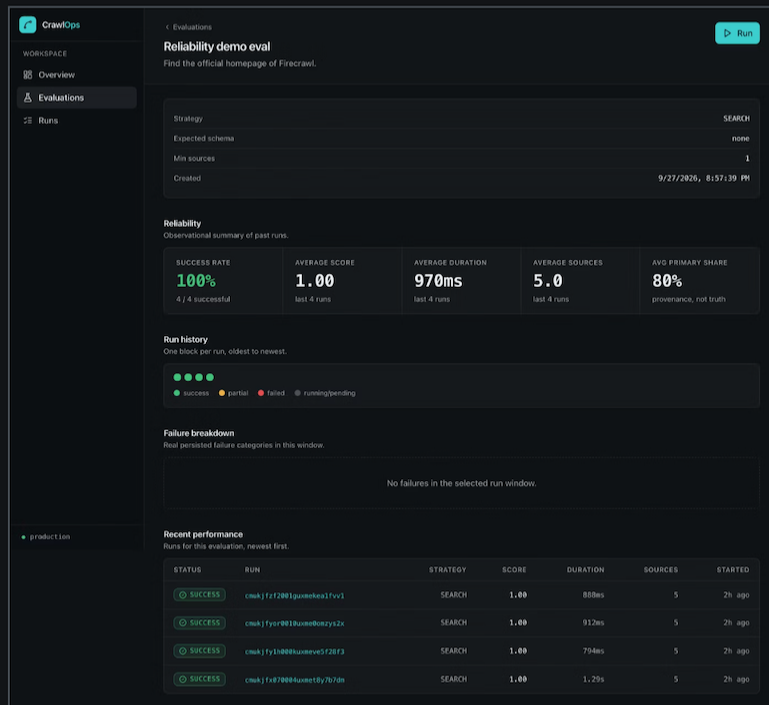
ONE SUCCESSFUL RUN ISN'T ENOUGH

4 / 4

Example production history

- **4 successful runs out of 4** in this example production history
- **Observational metrics:** Average score, average duration, source metrics, and failure breakdown
- **Diagnostic visibility:** Per-evaluation run history and failure breakdown

DATA LOCALITY: Analytics use already-persisted runs — no extra Firecrawl calls. Example only; not a benchmark or statistically significant.



FIRECRAWL DOES THE RESEARCH. CRAWLOPS TESTS THE WORKFLOW.

Separate provider success from application correctness:

An API 200 does not guarantee contract satisfaction.

Make failures observable and reproducible:

Persist attempts, structured output, source URLs, evaluator checks, and failure categories.

Measure reliability across repeated runs, not one demo:

Surface fragility across iterations before production release.

I built CrawlOps to explore the reliability layer around production web-research agents.

STACK: TypeScript · React · Fastify · PostgreSQL · Prisma · Firecrawl · Docker · Azure